

VERITRUST

Four Core Detection Architecture

Industry-Grade Implementation Blueprint

Scope	Link Detection Phishing Detection Deepfake Detection Unified Gateway
Architecture target	High assurance, evidence-first, reproducible, failure-aware
Source baseline	VeriTrust Deep Repository, Product, Security & Competitive Audit - 9 Aug 2026
Document date	9 August 2026

Architecture principle: detectors produce evidence; only the Unified Gateway produces the final trust decision.

0. Purpose, Scope and Trust Standard

This document intentionally designs only four core systems. Everything else is deferred.

Important accuracy statement

No probabilistic security or AI detector can be guaranteed 100% correct. The target here is high assurance: explicit uncertainty, calibrated evidence, fail-safe states, reproducibility, isolation of hostile processing, measurable release gates, and human-review paths for ambiguous outcomes.

In scope

Core system	What this blueprint defines
Link Detection	URL normalization, static heuristics, ML adapter, isolated live enrichment, sandbox evidence, link evidence output.
Phishing Detection	Text/email parsing, authentication evidence, content intelligence, cross-engine linkage, relationship checks, phishing evidence output.
Deepfake Detection	Image validation, quality/OOD gate, deterministic preprocessing, model registry, calibrated inference, disagreement/abstention, evidence output.
Unified Gateway	Orchestration, common evidence contract, Evidence Graph, correlation, policy, final decision, persistence and observability contracts.

Explicitly out of scope for this iteration

- C2PA/provenance, QR/quishing, audio/video deepfake models, campaign threat memory, browser extension, Gmail/Outlook integrations, SIEM/SOAR, enterprise SSO/SCIM, and learning-system integration.
- These can attach later through the same EvidenceAdapter interface without redesigning the four-core architecture.

Audit-grounded starting point

- Image synthetic/deepfake analysis is implemented, but provider label semantics and calibration need strengthening.
- Phishing/message analysis is implemented as classifier + heuristics + evidence/risk output.
- Link intelligence is implemented but lacks deep live enrichment and sandbox/reputation depth.
- The current Gateway has useful orchestration/policy foundations, but its correlation is score/rule aggregation rather than relationship-based evidence reasoning.
- Behavioral tests, benchmark/calibration artifacts and reproducible database migrations are release-quality gaps that must be closed before strong trust claims.

1. Architecture at a Glance

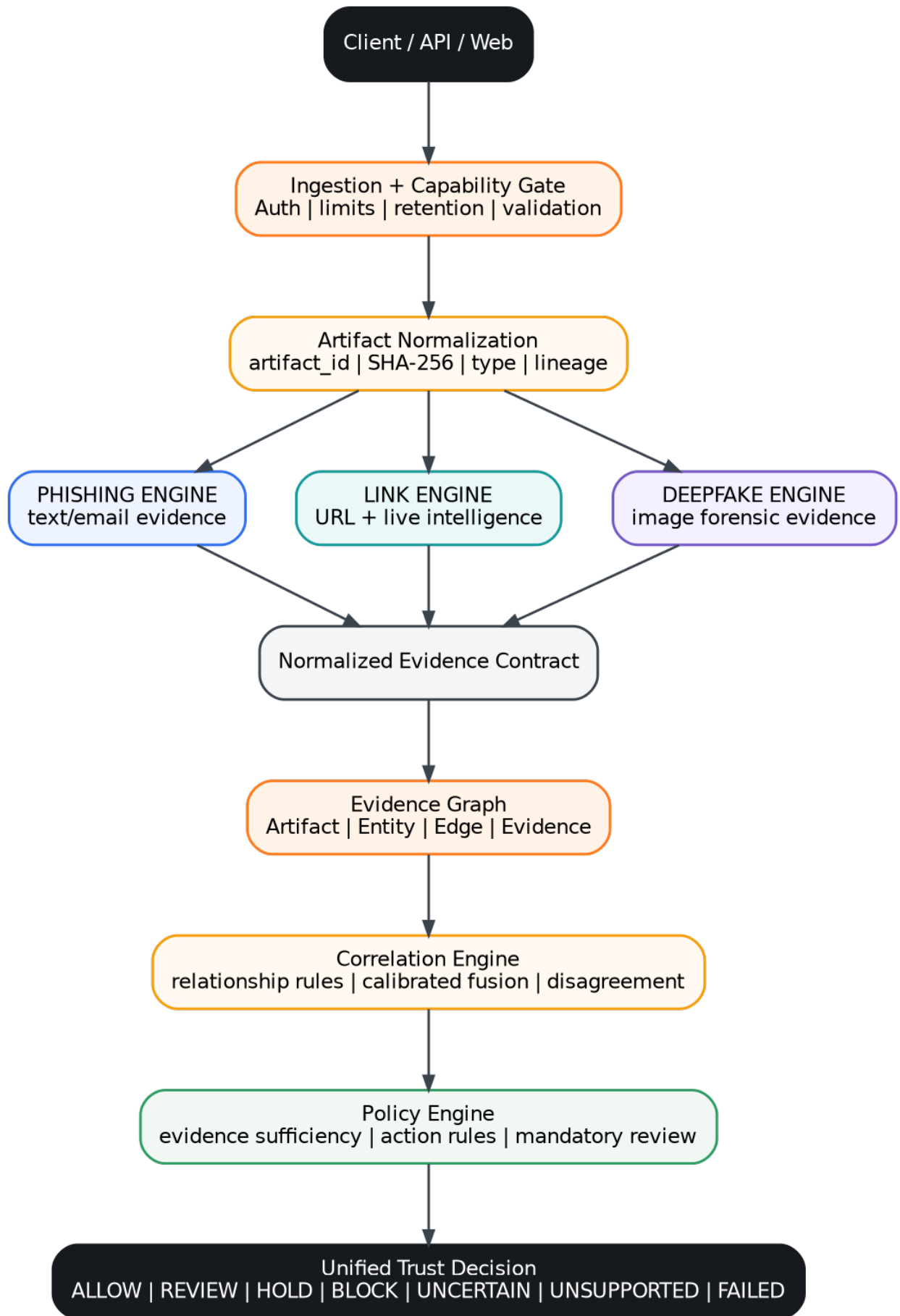
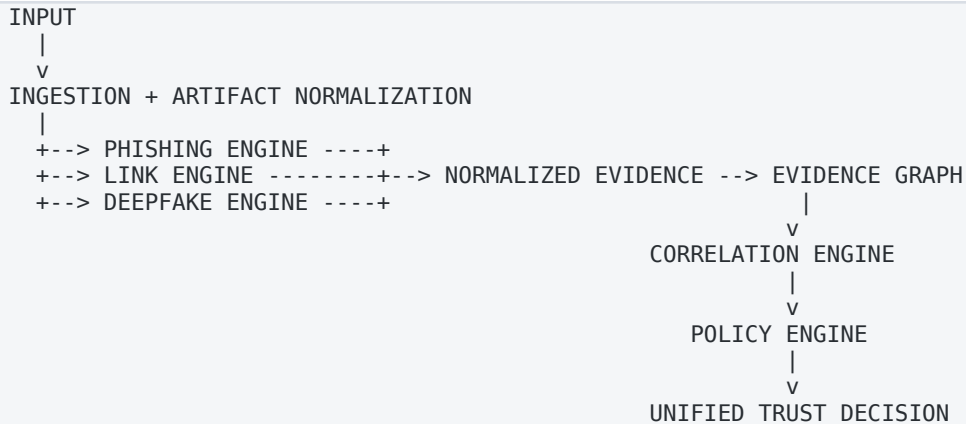


Figure 1. VeriTrust four-core target architecture.

Core invariant

Specialist engines never directly decide ALLOW/BLOCK. They emit typed evidence. The Unified Gateway owns correlation, policy evaluation, uncertainty and the final decision.

**Ten non-negotiable architecture rules**

1. Every submitted object receives an immutable artifact_id and SHA-256 before analysis whenever bytes are available.
2. Every model/provider result is versioned and traceable; unknown output schemas fail closed instead of being guessed.
3. Raw model score, calibrated probability, correlated risk and decision confidence remain separate fields.
4. A specialist engine emits Evidence; only the Gateway emits the final Decision.
5. Uncertainty is a first-class state. UNCERTAIN, UNSUPPORTED and FAILED are valid outcomes, not errors to hide.
6. Hostile URL execution happens in an isolated worker with network and resource controls, not in the ordinary API process.
7. Phishing email authentication is treated as evidence, not proof that a message is benign or malicious.
8. Model disagreement is preserved. Never average away conflicts without surfacing them.
9. Every decision references the exact artifact, evidence, model, correlation and policy versions used.
10. No production promotion occurs without benchmark, security, resilience and reproducibility gates passing.

2. Shared Kernel: Contracts All Four Systems Must Use

2.1 Artifact contract

```

Artifact {
  artifact_id
  parent_artifact_id?
  type           // url | text | email | image
  source         // web | api | derived
  sha256?
  mime_type?
  byte_size?
  retention_class
  created_at
  schema_version
}

```

2.2 Evidence contract

```
Evidence {
  evidence_id
  artifact_id
  evidence_type
  source { adapter, provider?, model_key?, version }
  observation { verdict, raw_score?, calibrated_probability?, value? }
  quality { status, score?, reasons[] }
  reason_codes[]
  derived_from[]
  observed_at
  schema_version
}
```

2.3 Decision contract

```
Decision {
  decision_id
  investigation_id
  action // ALLOW | REVIEW | HOLD | BLOCK | UNCERTAIN | UNSUPPORTED | FAILED
  risk_level
  decision_confidence
  evidence_sufficiency
  rationale_codes[]
  evidence_ids[]
  model_versions[]
  correlation_version
  policy_version
  created_at
}
```

2.4 Model registry contract

Field	Requirement
model_key	Stable internal identifier; never use display names as the contract key.
provider + exact revision	Exact model/provider revision or hash used for the result.
task + input contract	Supported media/input schema and preprocessing requirements.
output_label_map	Explicit provider-label -> VeriTrust semantic mapping. Unknown labels are rejected.
preprocessing_version	Immutable version for decode/resize/normalize/crop behavior.
calibration_version	Version of calibration transform and threshold set.
benchmark_version	Evaluation manifest used to qualify this model for deployment.
deployment_status	disabled shadow canary production rolled_back.

3. Part I - Link Detection Engine

Objective: transform any URL into reproducible static, model and live-enrichment evidence without exposing the core platform to hostile destinations.

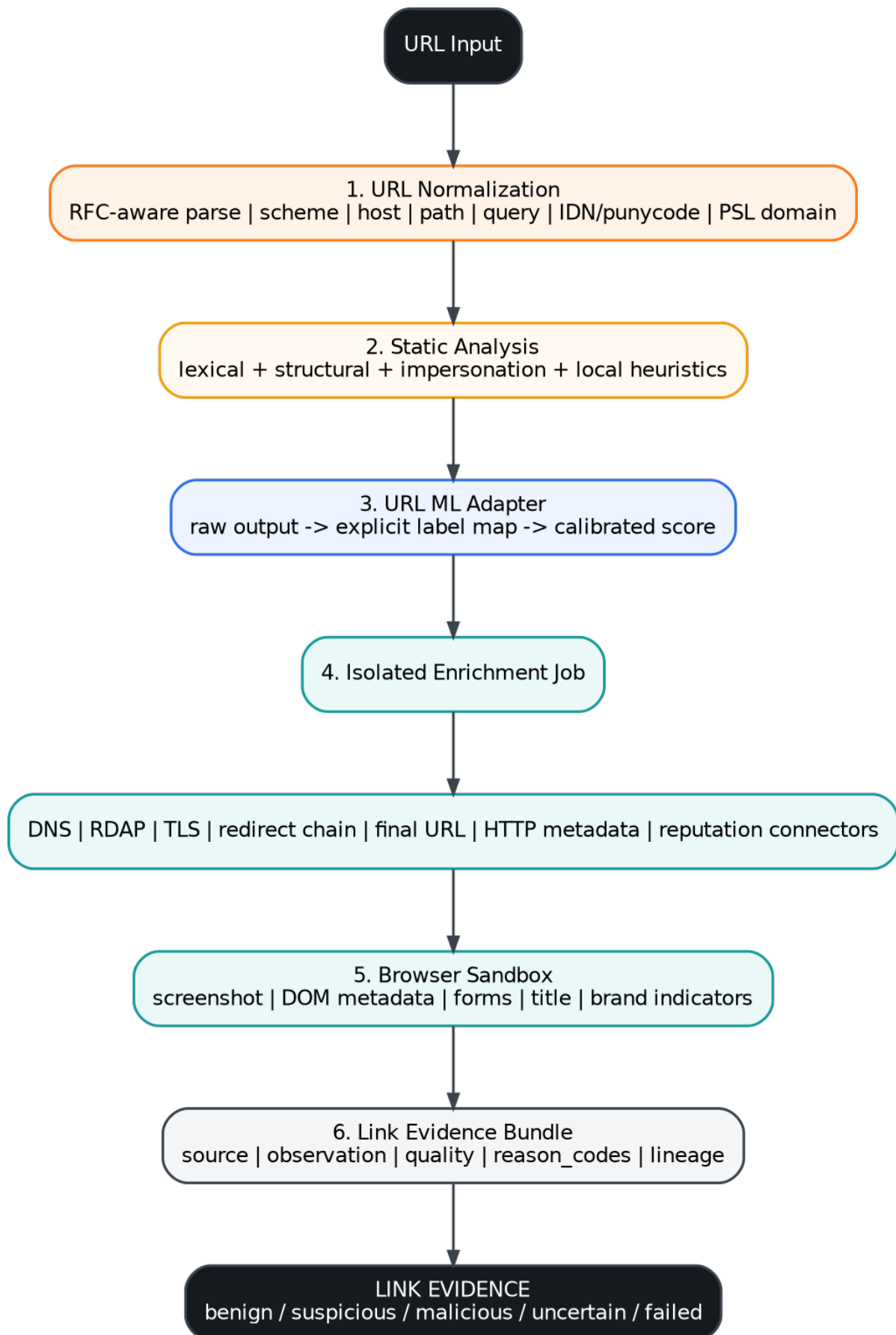


Figure 2. Link Detection industry-grade flow.

```

URL
|
+--> NORMALIZE / PARSE
|   +-- scheme / host / port / path / query
|   +-- Unicode <-> punycode
|   +-- PSL-aware registered domain
|
+--> STATIC ANALYSIS
|   +-- lexical/structural features
|   +-- impersonation/domain-similarity signals
|
+--> URL MODEL ADAPTER
|   +-- explicit output mapping
|   +-- calibrated probability
|
+--> ISOLATED LIVE ENRICHMENT
|   +-- redirects / final URL
|   +-- DNS / RDAP / TLS / HTTP metadata
|   +-- reputation connectors
|   +-- browser sandbox metadata
|
v
LINK EVIDENCE BUNDLE

```

3.1 Component responsibilities

Component	Implementation responsibility	Failure behavior
URL normalizer	Standards-aware parsing; scheme allowlist; percent-decoding safety; Unicode/punycode handling; PSL-aware registered domain extraction.	Malformed/unsupported URL -> UNSUPPORTED_INPUT; never crash the scan.
Static analyzer	Lexical and structural heuristics, suspicious encoding, domain/path/query features, brand/domain similarity signals.	Emit individual reason-coded evidence; do not convert heuristics directly into a final block.
URL ML adapter	Model request, exact label mapping, calibration, model/version recording.	Provider failure -> evidence status FAILED; Gateway decides whether remaining evidence is sufficient.
Enrichment worker	Redirect resolution, DNS/RDAP/TLS/HTTP metadata, reputation providers.	Timeouts and connector errors are isolated and visible; never silently become benign evidence.
Browser sandbox	Execute the destination in a constrained environment and collect non-sensitive analysis metadata.	Resource/time/network limits terminate the job; status becomes DEGRADED or FAILED.
Evidence builder	Normalize all link findings into the shared Evidence contract.	Schema failure blocks evidence ingestion and is observable.

3.2 URL safety boundary

Mandatory isolation

Any component that dereferences user-supplied URLs must run outside the normal application trust boundary. Validate every redirect hop, block non-web schemes, reject private/link-local/loopback destinations after DNS resolution, revalidate on redirects, apply strict time/byte limits, and restrict egress wherever practical.

- HTTP/HTTPS only for browser/enrichment execution unless a separately approved connector requires otherwise.
- Resolve and validate destination IPs before connection; reject loopback, private, link-local, multicast, metadata-service and other disallowed ranges.
- Repeat destination validation after redirects and DNS changes; do not trust the initially parsed hostname indefinitely.

- Use a dedicated worker identity with no access to application secrets, Supabase service credentials or internal admin endpoints.
- Set hard caps for redirects, response bytes, page execution time, DOM/screenshot artifact size and concurrent browser jobs.
- Store fetched artifacts according to retention policy; store evidence metadata separately from untrusted content.

3.3 Link evidence taxonomy

Evidence family	Example reason codes
Parsing / structure	MALFORMED_URL, NON_HTTP_SCHEME, ENCODED_HOST, UNUSUAL_PORT, SUSPICIOUS_PATH_STRUCTURE
Domain identity	REGISTERED_DOMAIN_MISMATCH, IDN_CONFUSABLE, BRAND_DOMAIN_SIMILARITY, PUBLIC_SUFFIX_PARSE_FAILURE
Network / registration	REDIRECT_CHAIN_RISK, NEW_OR_UNUSUAL_REGISTRATION, TLS_METADATA_ANOMALY, DNS_RESOLUTION_ANOMALY
Reputation	REPUTATION_MALICIOUS, REPUTATION_SUSPICIOUS, REPUTATION_NO_DATA
Browser	CREDENTIAL_FORM_PRESENT, BRAND_PAGE_MISMATCH, SCRIPTING_DEGRADED, SANDBOX_TIMEOUT
Model	URL_MODEL_HIGH_RISK, URL_MODEL_LOW_RISK, URL_MODEL_UNCERTAIN, URL_MODEL_FAILED

3.4 Link implementation sequence

11. Replace unsafe URL decoding with a total, exception-safe parser and introduce a PSL-aware registered-domain library.
12. Create LinkEvidenceAdapter implementing the common Evidence contract; preserve current heuristics as independent reason-coded signals.
13. Add explicit model output schemas and calibration metadata to the model registry.
14. Create an asynchronous URL enrichment job contract and queue.
15. Deploy the enrichment worker in an isolated runtime with SSRF/effective-egress controls.
16. Add DNS/RDAP/TLS/redirect/HTTP metadata collectors with per-source provenance and timeouts.
17. Add constrained browser execution as a separate stage, not inside the API handler.
18. Add fuzz/property tests for malformed URLs, IDNs, ccTLD/public-suffix cases, redirect chains and DNS rebinding scenarios.
19. Add connector outage and timeout tests proving that missing intel cannot silently yield SAFE.
20. Only after benchmark and security gates pass, allow live-enrichment evidence to influence production policy.

4. Part II - Phishing Detection Engine

Objective: combine message semantics, identity/authentication evidence and cross-engine signals without treating any single mechanism as proof of safety.

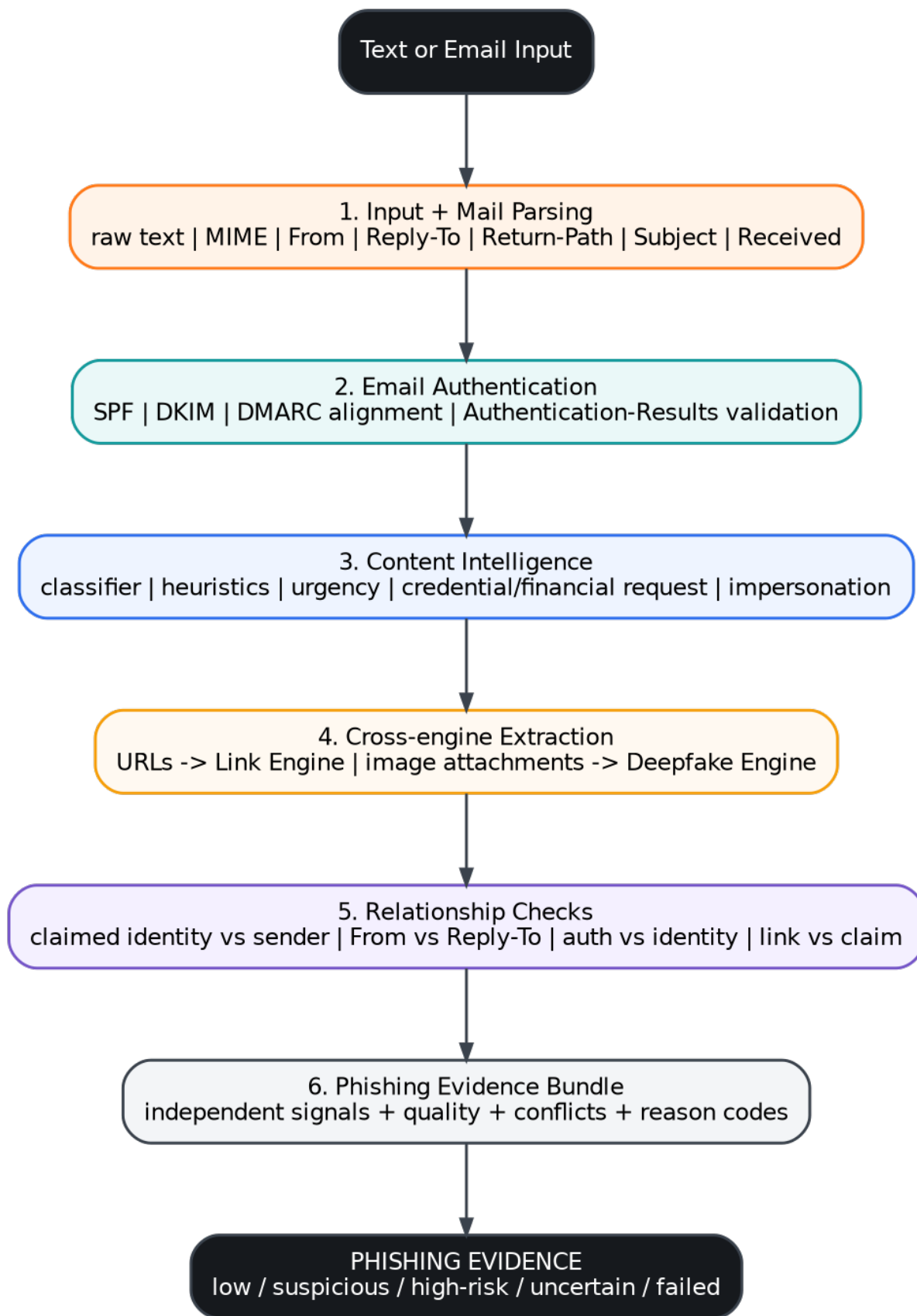


Figure 3. Phishing Detection industry-grade flow.

```

TEXT / EMAIL
|
+--> PARSE
|   +-- From / Reply-To / Return-Path / Subject / Received / MIME
|
+--> AUTHENTICATION EVIDENCE
|   +-- SPF
|   +-- DKIM
|   +-- DMARC alignment
|
+--> CONTENT INTELLIGENCE
|   +-- classifier
|   +-- heuristics
|   +-- urgency / credential / payment / impersonation signals
|
+--> CROSS-ENGINE ROUTING
|   +-- embedded URLs -> LINK ENGINE
|   +-- image attachments -> DEEPFAKE ENGINE when requested/supported
|
+--> RELATIONSHIP CHECKS
|   +-- claimed identity vs sender
|   +-- From vs Reply-To
|   +-- auth result vs author domain
|   +-- link destination vs message claim
|
|
v
PHISHING EVIDENCE BUNDLE

```

4.1 Input modes

Mode	Minimum processing	Trust limitation
Plain message text	Current classifier + heuristics + extracted URLs.	No sender/authentication claim can be made without headers.
Structured email / .eml	MIME/header parse + authentication evidence + text/content analysis + embedded-link routing.	Authentication results are evidence about domain use, not proof the message is harmless.

4.2 Email parser requirements

- Parse raw MIME using a maintained parser; preserve the original message bytes or immutable hash for reproducibility.
- Extract From, Reply-To, Return-Path, Subject, Message-ID, Received chain and Authentication-Results while preserving raw values.
- Treat malformed/ambiguous headers as explicit quality evidence rather than guessing.
- Apply bounded parsing: size limits, MIME depth limits, attachment count/size limits and time budgets.
- Never trust a client-supplied Authentication-Results header unless provenance establishes that it came from a trusted receiver boundary.

4.3 Authentication evidence rules

Signal	How VeriTrust should use it
SPF	Record pass/fail/neutral/softfail/temperror/pererror plus validated identity. Do not equate SPF pass with safe.
DKIM	Record signature verification result, signing domain and selector; keep multiple signatures separately.
DMARC	Evaluate author-domain alignment with authenticated SPF/DKIM identifiers; keep pass/fail and policy context.
Alignment	Model strict/relaxed alignment explicitly; do not reduce it to a generic "email verified" badge.
Authentication provenance	Record whether results were independently evaluated by VeriTrust or imported from a trusted receiver.

4.4 Content intelligence

- Keep existing classifier and heuristics, but emit each as independent evidence rather than a single opaque score.
- Use explicit semantic categories such as credential solicitation, payment request, urgent action, impersonation, account threat, reward/refund lure and link-click pressure.
- If an AI/ML classifier is used, store exact model and calibration versions just like the deepfake engine.
- Do not allow a language model summary, if added later, to directly set the final action; final policy remains evidence-driven.

4.5 Phishing implementation sequence

21. Wrap the existing classifier/heuristics in PhishingEvidenceAdapter and standardize reason codes.
22. Introduce robust plain-text vs full-email input contracts so the UI/API cannot imply header-authentication analysis when only text was supplied.
23. Add bounded MIME/.eml parsing and original-message hashing.
24. Implement SPF, DKIM and current DMARC validation/alignment as independent evidence sources.
25. Route extracted URLs to the Link Engine and merge their evidence by artifact relationship rather than copying link scores into phishing code.
26. Route supported image attachments to the Deepfake Engine only when the request/policy asks for media analysis.
27. Add identity relationship rules: From/Reply-To mismatch, claimed brand/domain mismatch and authentication/author-domain consistency.
28. Calibrate the phishing classifier on a held-out, current corpus; create language/content slices relevant to intended users.
29. Add malformed-MIME, spoofed-header, authentication-edge-case and indirect-mail regression fixtures.
30. Require Gateway-level evidence sufficiency before a high-impact phishing action can be issued.

5. Part III - Deepfake Detection Engine

Objective: produce scientifically defensible image-forensic evidence with explicit model semantics, calibration and abstention.

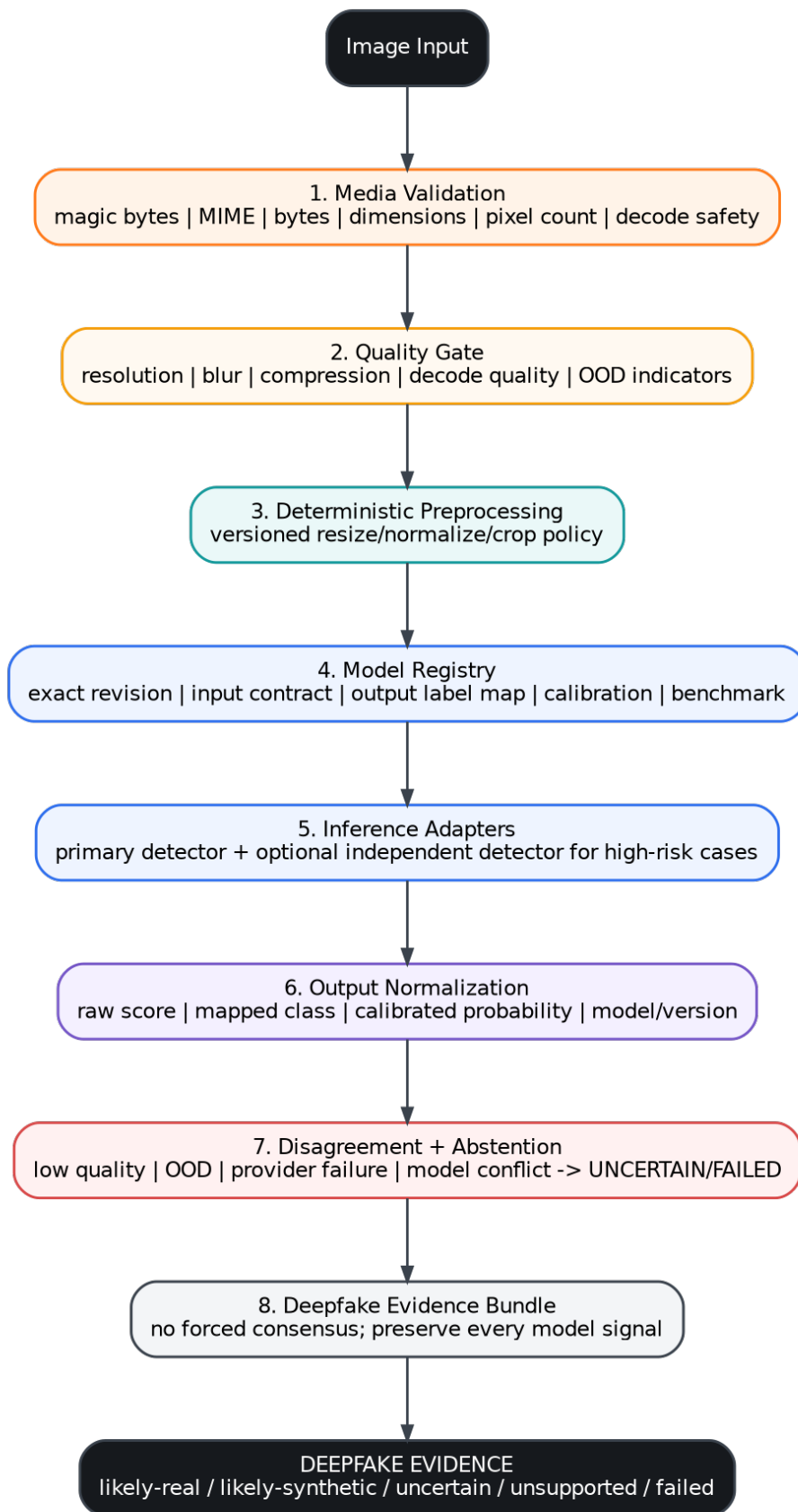


Figure 4. Image Deepfake Detection industry-grade flow.

```

IMAGE
|
+--> MEDIA VALIDATION
|   +-- magic bytes / MIME / byte size
|   +-- dimensions / pixel count / decode safety
|
+--> QUALITY + OOD GATE
|   +-- blur / resolution / compression / decode quality
|
+--> VERSIONED PREPROCESSING
|
+--> MODEL REGISTRY
|   +-- exact revision
|   +-- explicit output-label map
|   +-- calibration + benchmark version
|
+--> INFERENCE
|   +-- primary model
|   +-- optional independent model for high-risk cases
|
+--> NORMALIZE / CALIBRATE
|
+--> DISAGREEMENT + ABSTENTION
|   +-- low quality / OOD / provider failure / conflicting models
|
v
DEEPPFAKE EVIDENCE BUNDLE

```

5.1 Scientific contract

Never infer model semantics

The current audit found a fallback that can infer “fake” from unknown labels. Replace that behavior completely. Every production model must have an explicit task/input/output schema and label map. Unknown or changed provider output must produce `UNSUPPORTED_MODEL_OUTPUT` or `FAILED`, never a guessed class.

Layer	Required behavior
Media validation	Verify bytes/MIME; enforce dimensions and total pixel count before expensive decode/inference.
Quality gate	Measure input conditions that can invalidate model assumptions; quality affects evidence sufficiency.
Preprocessing	Deterministic and versioned. Same artifact + same version must produce the same model input.
Inference	Adapter-based; provider/model can be replaced without changing the Gateway contract.
Calibration	Raw score is not a real-world probability. Apply a versioned calibration transform only when validated.
Robustness	Benchmark re-encoding, resize/crop, screenshot/re-encode, blur/noise and common platform transformations.
OOD/abstention	Unknown content regime, low-quality input, provider failure or model disagreement must be surfaced.

5.2 Recommended decision vocabulary

Evidence verdict	Meaning
LIKELY_REAL	Validated evidence leans toward real under the qualified model/benchmark scope.
LIKELY_SYNTHETIC	Validated evidence leans toward synthetic/manipulated under the qualified scope.
UNCERTAIN	Evidence is conflicting, weak, OOD or below sufficiency

	requirements.
UNSUPPORTED	Input/model/task is outside the declared capability contract.
FAILED	Processing or provider failed; no semantic verdict should be inferred.

5.3 Calibration and benchmark package

- Frozen evaluation manifest: dataset version/hash, license, sampling, manipulation methods, content slices and real-content provenance.
- Metrics: AUROC/AUPRC plus precision, recall, false-positive rate and false-negative rate at each production threshold.
- Calibration: reliability curve, Brier score/calibration error, confidence intervals and versioned transformation.
- Robustness: compression/transcoding, resize/crop, screenshot/re-encode, blur/noise, overlays and common social-platform transformations.
- OOD/abstention: unfamiliar generators/content regimes, low quality, failed preprocessing, provider outage and model disagreement.
- Release delta: no model or threshold promotion without comparing against the currently deployed version and recording rollback metadata.

5.4 Deepfake implementation sequence

31. Create the versioned ModelRegistry and remove all heuristic fallback label interpretation.
32. Add dimension/pixel-count and bounded-decode guards before model preprocessing.
33. Version the preprocessing pipeline and include preprocessing_version in every evidence object.
34. Build a frozen benchmark harness and baseline the currently deployed provider/model.
35. Calibrate thresholds only after held-out evaluation; until then, label scores as model scores rather than probabilities.
36. Introduce quality/OOD evidence and make it part of Gateway sufficiency checks.
37. For high-impact cases, allow a second independent detector through the same adapter interface; preserve disagreement.
38. Implement UNCERTAIN/UNSUPPORTED/FAILED as first-class result states.
39. Add robustness regression suites and provider-response fixtures.
40. Deploy model changes through shadow -> canary -> production with instant rollback to a previous registry version.

6. Part IV - Unified Gateway

Objective: make the Gateway the only authoritative decision boundary while preserving every underlying signal and its uncertainty.

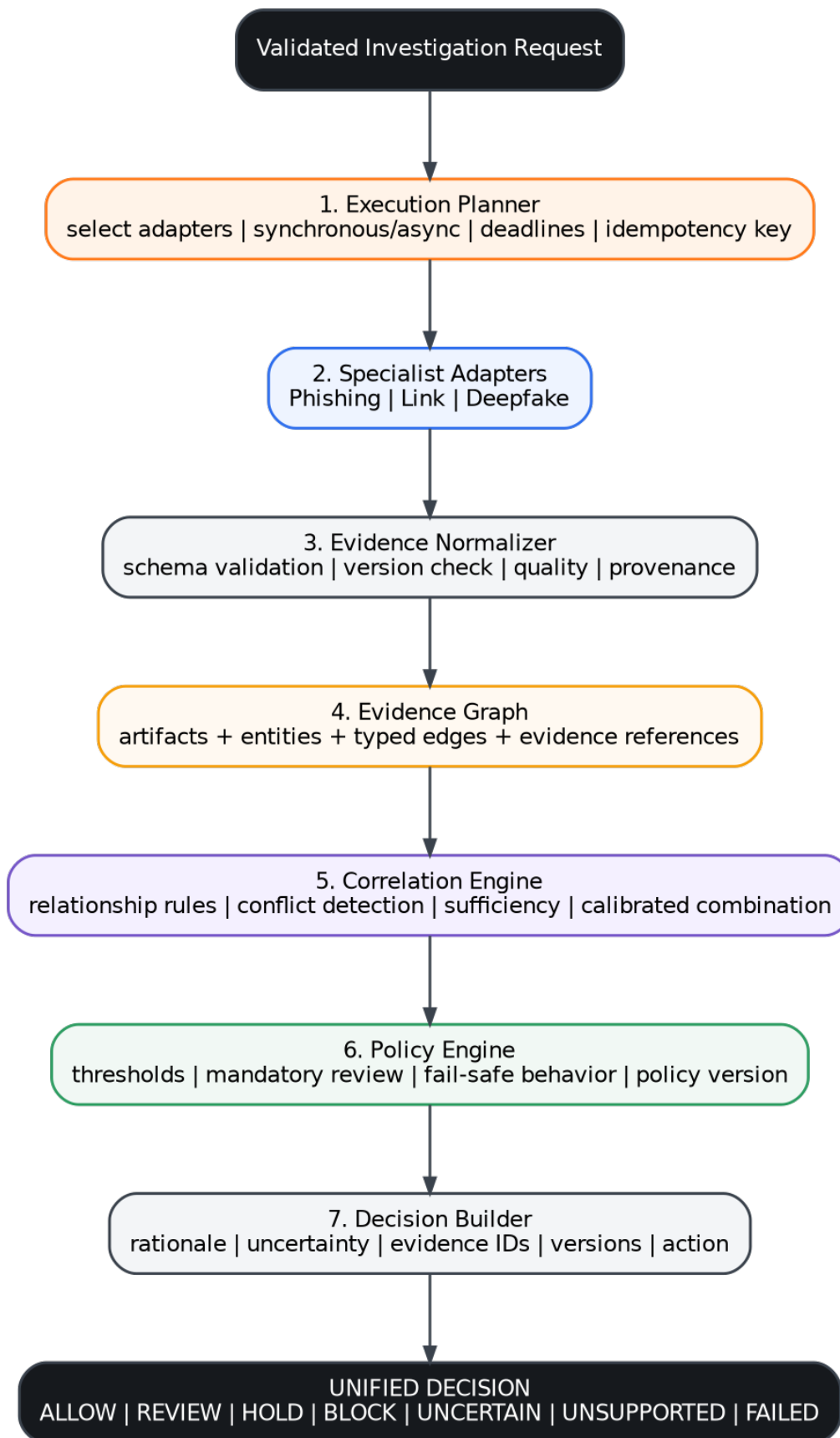


Figure 5. Unified Gateway decision pipeline.

```

VALIDATED REQUEST
|
+--> EXECUTION PLAN
|   +-- select Phishing / Link / Deepfake adapters
|   +-- deadlines / async jobs / idempotency
|
+--> NORMALIZED EVIDENCE
|
+--> EVIDENCE GRAPH
|   +-- Artifact
|   +-- Entity
|   +-- Edge
|   +-- Evidence
|
+--> CORRELATION
|   +-- relationship rules
|   +-- disagreement/conflict
|   +-- evidence sufficiency
|   +-- calibrated combination where justified
|
+--> POLICY
|   +-- thresholds
|   +-- mandatory review
|   +-- fail-safe behavior
|
+--> DECISION BUILDER
|   +-- action / risk / confidence / reasons / versions
|
v
UNIFIED TRUST DECISION

```

6.1 Execution planner

- Validate request against a single server capability registry before upload/processing.
- Create investigation_id and idempotency key; repeated submissions with the same key must not create conflicting duplicate jobs.
- Select only applicable adapters; unsupported modalities are rejected or explicitly marked UNSUPPORTED before expensive work.
- Separate fast synchronous evidence from async enrichment. The client receives durable job status rather than holding a long request open indefinitely.
- Set per-adapter deadlines and cancellation semantics. One slow provider must not stall the whole platform.

6.2 Evidence Graph MVP

```

Artifact(email_01)
|-- contains --> Artifact(url_01)
|-- contains --> Artifact(image_01)

Entity(sender_domain) -- claimed_identity_of --> Entity(brand)
Entity(url_domain)    -- destination_for -----> Artifact(url_01)

Evidence(phish_01)    -- supports -----> Decision(candidate)
Evidence(link_07)     -- supports -----> Decision(candidate)
Evidence(df_03)       -- conflicts/qualifies -> Decision(candidate)

```

- Use relational Postgres tables for the MVP; a dedicated graph database is not required initially.
- Edges must be typed and point back to source_evidence_id. A graph without provenance is only a visualization.
- Never overwrite evidence. New model runs create new immutable evidence records and new Decision versions.

6.3 Correlation engine rules

Rule class	Behavior
Independent evidence	Preserve each specialist verdict, score, quality and

	version.
Reinforcement	Raise case-level risk only when a versioned correlation rule is supported by evidence relationships.
Conflict	If strong evidence disagrees, lower decision confidence and/or require REVIEW rather than averaging to certainty.
Sufficiency	A decision can require minimum evidence classes/quality before BLOCK or ALLOW is permitted.
Missing evidence	Provider outage, timeout or unsupported input is not negative evidence and must not be treated as safe.
Versioning	Every decision stores correlation_version so historical results remain reproducible.

6.4 Policy engine

Policy owns action; models do not

A high model score may contribute to risk, but only a versioned policy may authorize ALLOW, REVIEW, HOLD or BLOCK. This keeps operational behavior separate from model semantics and makes policy changes auditable.

Example policy logic (conceptual):

```
IF evidence_sufficiency < REQUIRED
  -> REVIEW or UNCERTAIN
ELSE IF high_risk_link AND phishing_identity_conflict
  -> BLOCK
ELSE IF deepfake_models_disagree AND media_is_material_to_case
  -> REVIEW
ELSE
  -> evaluate remaining versioned policy rules
```

NOTE: thresholds must come from calibration/validation, not arbitrary percentages.

6.5 Final states

State	Operational meaning
ALLOW	Policy permits the action based on sufficient evidence; not equivalent to “guaranteed safe”.
REVIEW	Evidence exists but human/analyst review is required by policy.
HOLD	Processing/policy requires temporary non-allow while more evidence or async work completes.
BLOCK	Policy prohibits the action based on sufficient high-risk evidence.
UNCERTAIN	Evidence quality, conflict or OOD prevents a defensible directional verdict.
UNSUPPORTED	Requested capability is outside the declared capability registry.
FAILED	Processing failed; system must not infer a benign result.

7. Minimal Data Model and API Surface

7.1 Recommended relational tables

Table	Key fields / purpose
investigations	investigation_id, workspace/user scope, status, requested_adapters, created_at, completed_at.

artifacts	artifact_id, parent_artifact_id, type, sha256, MIME/size, source, retention_class.
entities	entity_id, type, normalized_key. Examples: domain, email address, brand/identity claim.
edges	from_entity/artifact, relation, to_entity/artifact, source_evidence_id, observed_at, quality.
evidence	evidence_id, artifact_id, adapter, provider/model/version, observation, quality, reason_codes, raw reference.
decisions	decision_id, investigation_id, action, risk, confidence, sufficiency, evidence_ids, policy/correlation versions.
model_versions	model registry and deployment/calibration/benchmark metadata.
policy_versions	immutable policy definitions and promotion/rollback metadata.
jobs	async execution state, attempt, deadline, cancellation, idempotency and error class.

7.2 Suggested API contracts

Endpoint	Purpose
POST /api/investigations	Create one investigation with text/email/URL/image artifacts and requested adapters.
GET /api/investigations/{id}	Return durable status, progress, current decision and evidence summaries.
POST /api/investigations/{id}/cancel	Cancellation request for async work.
GET /api/system/capabilities	Single source of truth for supported inputs, limits, adapters and degraded/maintenance state.
GET /api/models	Operator/admin view of model registry status; not necessarily public.

8. Robustness, Security and Trust Controls

8.1 Fail-safe matrix

Failure	Required behavior	Forbidden behavior
Unknown model label	Reject mapping; FAILED/UNSUPPORTED evidence; alert.	Guess label semantics.
Model provider outage	Mark evidence FAILED; continue only if policy allows with sufficient independent evidence.	Treat missing model score as low risk.
URL enrichment timeout	DEGRADED/FAILED link evidence; Gateway may HOLD/REVIEW.	Return benign because no threat was observed.
DNS changes / redirect to private IP	Abort dereference and flag security event.	Follow destination without revalidation.
Malformed email/MIME	Bound parser; emit quality failure; preserve hash.	Crash worker or silently discard malformed portions.
Low-quality/OOD image	UNCERTAIN or REVIEW depending on policy.	Force real/fake class.
Conflicting strong evidence	Reduce confidence and surface disagreement.	Blind average into a confident decision.
Database write partial failure	Atomic transaction or idempotent retry.	Expose decision that cannot be reproduced from persisted evidence.

8.2 Security controls

- Tenant/workspace authorization at every read/write boundary; test cross-tenant IDs explicitly.

- Hash API keys/secrets; never include raw credentials in evidence, logs or exports.
- Structured logging with request/investigation IDs, but redact message/media contents by default.
- File/image parsing limits and decompression-bomb guards.
- SSRF-resistant URL enrichment with restricted worker credentials and egress controls.
- Signed/validated provider callbacks where used; idempotent processing for all external events.
- Dependency and supply-chain checks in CI; reproducible install from lockfile; no vendored node_modules in release artifacts.
- Database migrations as source of truth; clean-room bootstrap and restore tests before production releases.

8.3 Observability

Metric family	Examples
Reliability	request failures, job retries, queue age, timeouts, degraded/failed rate, cancellation latency.
Providers	latency, error rate, throttling, schema-change alerts, model/version distribution.
Decisions	ALLOW/REVIEW/HOLD/BLOCK/UNCERTAIN/UNSUPPORTED/FAILED distribution by policy/model version.
Model validity	drift indicators, calibration monitoring, false-positive/false-negative feedback, slice performance.
Link worker	blocked SSRF attempts, redirect depth, browser crashes, DNS anomalies, resource-limit terminations.
Data integrity	orphan evidence, decision/evidence version mismatches, migration/schema hash mismatch.

9. Test Strategy and Release Gates

9.1 Test pyramid

Layer	Minimum coverage
Unit	parsers, label maps, calibration transforms, reason-code rules, policy evaluation, graph edge creation.
Property / fuzz	URL parsing/encoding, MIME parsing, domain/public-suffix handling, malformed provider outputs.
Contract	every EvidenceAdapter against versioned schemas and provider fixtures.
Integration	Supabase/Postgres, queues, workers, provider adapters, idempotency and retries.
Security regression	tenant isolation, SSRF/rebinding, API scopes, upload bounds, signed events, XSS-safe rendering.
Browser E2E	submit -> progress -> result -> degraded/failed flows for the four-core product journeys.
Model benchmark	held-out metrics, calibration, robustness, OOD/abstention, regression vs deployed model.
Resilience	provider outage, DNS failure, queue delay, database retry, worker crash, cancellation and rollback.
Load	API throughput, queue saturation, browser-worker concurrency, database contention and backpressure.

9.2 Non-negotiable release gates

41. Model contract gate: every production model has an explicit label map; unknown output can never produce a semantic verdict.
42. Calibration gate: any field called probability/confidence must have validation evidence appropriate to that meaning.

43. Capability truth gate: UI, API and backend enforcement use the same capability/limit definition.
44. Security gate: tenant/RLS tests, SSRF corpus, file bounds, API-key scope and dependency checks pass.
45. Migration gate: a clean database can be built from versioned migrations and upgraded/restored in CI.
46. Workflow gate: any final decision can be reproduced from artifact/evidence/model/correlation/policy versions.
47. Resilience gate: provider outages and timeouts yield explicit degraded states, never false-safe behavior.
48. Benchmark regression gate: model/threshold changes meet documented deltas and have a rollback target.
49. Observability gate: production dashboards/alerts exist for failures, provider drift and decision distribution.

10. Implementation Roadmap - Exact Order of Work

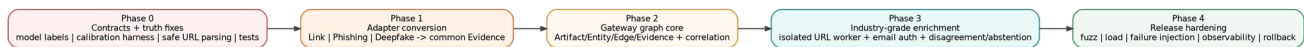


Figure 6. Recommended implementation order.

Phase 0 - Truth and contract foundation

- Create shared Artifact, Evidence, Decision, ModelVersion and PolicyVersion schemas.
- Create /api/system/capabilities and drive accepted inputs/limits from it.
- Fix deepfake label normalization; reject unknown model labels.
- Add safe URL decoding and PSL-aware registered-domain parsing.
- Add image dimension/pixel-count guards.
- Create governed database migrations and behavioral test harness.

Phase 1 - Convert current detectors into EvidenceAdapters

- PhishingEvidenceAdapter wraps current text classifier/heuristics.
- LinkEvidenceAdapter wraps current parsing/heuristics/model path.
- DeepfakeEvidenceAdapter wraps provider inference using explicit registry semantics.
- Gateway accepts only validated Evidence objects; old detector-specific result shapes become compatibility wrappers, not the internal truth.

Phase 2 - Evidence Graph + Gateway correlation

- Add artifacts/entities/edges/evidence/decisions tables and immutable version references.
- Create graph-building rules from parsed relationships.
- Replace max-score/bonus correlation with versioned relationship rules, conflict detection and evidence sufficiency.
- Move final action authority into PolicyVersion.

Phase 3 - Industry-grade depth inside the four systems

- Link: isolated live enrichment and browser sandbox.
- Phishing: full .eml/MIME parsing + SPF/DKIM/current DMARC alignment.
- Deepfake: benchmark/calibration + quality/OOD + multi-model disagreement/abstention.
- Gateway: async execution planner, retries/cancellation, durable progress and explicit degraded states.

Phase 4 - Release hardening

- Fuzz and security regression corpus.
- Failure injection and provider outage drills.
- Load/backpressure validation for queue and browser workers.
- Model/policy shadow and canary rollout with rollback.
- Production observability and decision-drift dashboards.
- Clean-room deployment and restore rehearsal.

11. First 24 Engineering Tickets

ID	Area	Deliverable
VT2-001	Shared schemas	Define versioned Artifact/Evidence/Decision JSON schemas + runtime validators.
VT2-002	Capabilities	Implement /api/system/capabilities and derive input accept/limits from server truth.
VT2-003	Deepfake correctness	Create model registry and remove unknown-label fallback semantics.
VT2-004	Link correctness	Implement safe URL decode/parser and PSL-aware registered domain.
VT2-005	Media safety	Add image dimensions/pixel/decode budget guards.
VT2-006	Migrations	Move schema/RPC changes into executable version-controlled migrations.
VT2-007	Tests	Introduce unit/contract/integration/E2E test stack in CI.
VT2-008	Adapters	Implement PhishingEvidenceAdapter.
VT2-009	Adapters	Implement LinkEvidenceAdapter.
VT2-010	Adapters	Implement DeepfakeEvidenceAdapter.
VT2-011	Graph	Create artifacts/entities/edges/evidence/decisions persistence.
VT2-012	Gateway	Create evidence normalizer + schema/version rejection.
VT2-013	Correlation	Create correlation v1 with relationship rules/conflict/sufficiency.
VT2-014	Policy	Create immutable PolicyVersion and action builder.
VT2-015	Jobs	Create durable async job/idempotency/cancellation contract.
VT2-016	Link worker	Deploy isolated URL-enrichment worker and SSRF controls.
VT2-017	Link intel	Add redirect/DNS/RDAP/TLS/HTTP/reputation evidence.
VT2-018	Link sandbox	Add constrained browser sandbox stage.
VT2-019	Email	Add bounded .eml/MIME parser and original-message hashing.
VT2-020	Email auth	Implement SPF/DKIM/DMARC evidence and alignment.
VT2-021	Deepfake benchmark	Build held-out evaluation/calibration/robustness harness.
VT2-022	Deepfake abstention	Implement quality/OOD/disagreement states.
VT2-023	Observability	Add provider/job/decision/model-version dashboards and alerts.
VT2-024	Release gates	Automate security, migration, benchmark, resilience and reproducibility gates.

12. Definition of Done for the Four-Core Architecture

- Link Detection can parse malformed/IDN/public-suffix URLs safely, enrich hostile URLs in isolation, preserve connector provenance and return explicit degraded/failed states.
- Phishing Detection can clearly distinguish plain-text analysis from full-email analysis, validate SPF/DKIM/DMARC evidence, route URLs/images to specialist engines and preserve identity/authentication relationships.
- Deepfake Detection has explicit per-model semantics, versioned preprocessing, held-out benchmark/calibration artifacts, robustness tests and an abstention path for weak/OOD/conflicting evidence.
- Unified Gateway is the sole final decision authority and can reproduce every decision from immutable artifact/evidence/model/correlation/policy versions.
- No missing provider, unsupported modality, parser failure or model disagreement can silently degrade into ALLOW/SAFE.
- All four systems use the same capability truth source, evidence schema, error taxonomy, observability context and release gates.
- Security and correctness regression tests run in CI; database bootstrap/upgrade/restore is reproducible from source.
- Production dashboards expose provider errors, queue health, degraded/failed scans, decision distribution, model versions and drift/feedback signals.
- The UI explains why a decision was made using evidence/reason codes rather than a single unexplained AI confidence percentage.

North-star definition

VeriTrust is ready to claim a high-assurance architecture when the four specialist and gateway flows are measurable, reproducible, failure-aware and honest about uncertainty - not when they merely return a result for every input.

13. Source Basis and Standards References

Primary project source

- VeriTrust Deep Repository, Product, Security & Competitive Audit - Architecture truth, capability table, findings VT-01/02/03/04/05/06/07/11/15, model-validity review, target architecture, roadmap and release gates (9 August 2026).

External standards used to strengthen the implementation blueprint

- NIST AI Risk Management Framework (AI RMF 1.0) - <https://www.nist.gov/itl/ai-risk-management-framework>
- NIST SP 800-218, Secure Software Development Framework (SSDF) v1.1 - <https://csrc.nist.gov/pubs/sp/800/218/final>
- OWASP Server-Side Request Forgery Prevention Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
- RFC 3986 - Uniform Resource Identifier (URI): Generic Syntax - <https://www.rfc-editor.org/info/rfc3986/>
- RFC 7208 - Sender Policy Framework (SPF) - <https://www.rfc-editor.org/info/rfc7208/>
- RFC 6376 - DomainKeys Identified Mail (DKIM) - <https://www.rfc-editor.org/info/rfc6376/>
- RFC 9989 - Domain-Based Message Authentication, Reporting, and Conformance (DMARC), Proposed Standard, May 2026 - <https://www.rfc-editor.org/info/rfc9989/>

Standards note

The external references are implementation guidance and protocol standards. They do not certify VeriTrust. Certification/assurance would require separate operational, security and model-validation evidence against the deployed system.

END OF BLUEPRINT